УНИВЕРСИТЕТ ИТМО

Домашняя работа №2

Проектирование и технический дизайн API

Выполнил:	Камалов Руслан Олегович
Группа:	БР1.2
ИСУ:	408732

Санкт-Петербург

2026

Отчёт по домашней работе №2

Студент: Камалов Руслан Олегович · Группа: БР1.2 · ИСУ: 408732

Цель

Я сформулировал требования к FitFlow и спроектировал REST API в формате OpenAPI 3.0.3.

Ход работы

Я определил роли гостя, пользователя и администратора, а также правила доступа к чужим и административным ресурсам. API включает регистрацию и вход, каталог тренировок, личные планы, историю и сводку прогресса, статьи и health-check. Для ошибок я задал единый объект `code/message/details`, а для защищённых операций - Bearer JWT.

Контракт находится в `docs/openapi.yaml`, требования - в `docs/homework-2-requirements.md`.

Спецификация успешно разбирается YAML-парсером и содержит 14 групп путей. Swagger UI подключён к монолиту по адресу `/api-docs`.

Вывод

Я получил формальный контракт, по которому можно независимо реализовать сервер и подготовить клиентские проверки.

Приложение

ДЗ 2. Требования и проектирование API FitFlow

Роли

- Гость: регистрация, вход, просмотр тренировок и опубликованных статей.
- Пользователь: возможности гостя, просмотр профиля, управление только своими планами и прогрессом.
- Администратор: возможности пользователя, управление тренировками и право изменять/удалять любые статьи.

Функциональные требования

- Регистрация по уникальному email и паролю не короче 8 символов.
- Вход возвращает JWT access token сроком на 2 часа.
- Каталог тренировок поддерживает фильтрацию по типу/уровню и пагинацию.
- Администратор выполняет CRUD тренировок.
- Пользователь выполняет CRUD личных планов и задаёт упорядоченный список тренировок.
- Пользователь создаёт и просматривает только собственные записи прогресса.
- Сводка прогресса содержит число выполнений и суммарную длительность.
- Все читают статьи по slug; авторизованный пользователь создаёт статью, а автор либо администратор изменяет и удаляет её.
- API возвращает единый формат ошибок и корректные HTTP-коды.
- `/health` доступен без авторизации.

Нефункциональные требования и целевые улучшения

- JSON REST API, контракт OpenAPI 3.0; HTTPS требует отдельной настройки TLS;
- stateless-аутентификация Bearer JWT;
- пароль хранится только как bcrypt-хеш;
- валидация входа, принцип наименьших привилегий, запрет доступа к чужим ресурсам;
- PostgreSQL и транзакция при замене состава плана;
- структурированные логи с маскированием секретов - целевое улучшение, сейчас используется console;
- целевое время ответа типового локального запроса до 500 мс; нагрузочным тестированием этот показатель не подтверждён;
- контейнерный запуск и конфигурация через переменные окружения.

Ресурсы и маршруты

Ресурс	Маршруты
Auth	`POST /api/v1/auth/register`, `POST /api/v1/auth/login`, `GET /api/v1/auth/me`
Workouts	`GET/POST /api/v1/workouts`, `GET/PATCH/DELETE /api/v1/workouts/{id}`
Plans	`GET/POST /api/v1/plans`, `GET/PATCH/DELETE /api/v1/plans/{id}`, `PUT /api/v1/plans/{id}/workouts`
Progress	`GET/POST /api/v1/progress`, `GET /api/v1/progress/summary`, `DELETE /api/v1/progress/{id}`
Articles	`GET/POST /api/v1/articles`, `GET /api/v1/articles/{slug}`, `PATCH/DELETE /api/v1/articles/{id}`

Полный машиночитаемый контракт: `openapi.yaml` (./openapi.yaml).

Коды ошибок

- `400 VALIDATION_ERROR` - некорректные данные;
- `401 UNAUTHORIZED` - токен отсутствует, истёк или неверен;
- `403 FORBIDDEN` - недостаточно прав либо чужой ресурс;
- `404 NOT_FOUND` - ресурс отсутствует;
- `409 CONFLICT` - конфликт уникальности;
- `500 INTERNAL_ERROR` - непредвиденная ошибка.